Learning By Project

DeepSight App

Applicazione di ricerca testuale di immagini e video da archivio privato.

Commessa: DAI252701
ID Gruppo: 02
Nome Gruppo: DS2
Versione: 1.0
Data: 20/07/2026

Membri del gruppo:
BARADELLO Fabio
CHRISTIAN Lovano
SAMUEL Rivoiro
FEDERICO Bertazzoni
SAMIA Jaouhar

Sommario

1. Analisi della realtà di riferimento 4
Clienti di riferimento 4
2. Funzionalità dell'interfaccia grafica 4
2.1 Home (Dashboard) 4
2.2 Caricamento (Upload) 4
2.3 Ricerca 5
3. Progettazione del database 5
Tabella file_multimediali (DuckDB) 6
Tabella tag_assegnati (DuckDB) 6
Collezione ChromaDB (multimedia_clip) 6
3.1 Schema della Logica di Funzionamento 7
Pipeline di Upload e Processing dei Dati 7
Algoritmo di Ricerca Avanzata (Ibrido) 7
4. Requisiti di Sistema 8
4.1 Piattaforma di sviluppo 8
Linguaggio e runtime 8
Interfaccia grafica 8
Persistenza dati 8
Librerie AI/ML e supporto 8
Ambiente di esecuzione e configurazione 8
4.2 Architettura software 9
Modelli di intelligenza artificiale utilizzati 9
4.3 Requisiti minimi hardware 10
5. Descrizione delle classi (OOP) 11
Database (in database.py) 11
DeepSightApp (in home.py) 11
UploadFrame (in upload.py) 11
RicercaFrame (in ricerca.py) 11
6. Flussi di funzionamento 12
6.1 Pipeline di Upload e processing 12
6.2 Ricerca avanzata (ibrida) 13
7. Deployment e installazione 14
Installazione (Windows) 14
Installazione (macOS / Linux) 14

Configurazione e primo avvio 14
Vincoli sulle versioni delle dipendenze 15
8. Documentazione delle API (funzioni e metodi) 16
8.1 config.py — Configurazione centralizzata 16
8.2 start_app.py — Bootstrap e verifica dipendenze 16
8.3 database.py — Classe Database 17
8.4 AI_engine.py — Motore AI (modulo funzionale) 19
8.5 home.py — Classe DeepSightApp 22
8.6 upload.py — Classe UploadFrame 24
8.7 ricerca.py — Funzioni di modulo e classe RicercaFrame 28
Funzioni di modulo 28
Metodi della classe RicercaFrame 28
9. Riferimenti 32

1. Analisi della realtà di riferimento

DeepSight è un applicativo desktop per la ricerca avanzata di foto e video in un archivio privato. L'utente aggiunge progressivamente nuovi contenuti che l'applicazione categorizza in automatico, ottimizzando le successive funzioni di ricerca.

La ricerca di immagini è tradizionalmente implementata tramite metadati ed etichette inserite manualmente: un processo lungo, dispendioso e potenzialmente impreciso. L'utente a cui si rivolge DeepSight non ha tempo né risorse per etichettare in modo efficiente tutte le proprie immagini. È qui che intervengono le funzionalità di intelligenza artificiale dell'applicazione, che in autonomia individuano il "significato" dell'immagine (o dei fotogrammi di un video) e abilitano una ricerca vettoriale più efficiente e precisa.

Clienti di riferimento

- **Content creator** — per archiviazione e ricerca di contenuti destinati alla pubblicazione su web, blog e piattaforme social.
- **Associazioni culturali o ricreative** — per l'archiviazione di contenuti legati a eventi e iniziative e la loro pubblicazione su sito o social.
- **Aziende private** — per la gestione on-premise dei propri contenuti e la pubblicazione su sito istituzionale e canali social.

L'interfaccia grafica intuitiva permette all'utente di effettuare caricamenti e ricerche in modo chiaro e fluido, senza alcuna conoscenza tecnica dei modelli AI sottostanti.

2. Funzionalità dell'interfaccia grafica

L'applicativo, realizzato in Python con la libreria CustomTkinter, adotta un'architettura a schermate multiple (frame) gestite da un unico controller centrale (la classe DeepSightApp). Sono presenti tre schermate principali — Home, Caricamento e Ricerca — raggiungibili da una barra di navigazione fissa presente in ogni pagina. Le schermate non vengono ricreate a ogni cambio pagina ma solo nascoste/mostrate, per cui la navigazione è istantanea.

2.1 Home (Dashboard)

Scopo. Permettere una rapida comprensione del funzionamento dell'app al primo utilizzo e tenere sotto controllo alcune statistiche e insight rilevanti.

Elementi principali:

- Barra di navigazione con logo (pulsante Home), pulsante Caricamento e pulsante Ricerca.
- Due card di accesso rapido (Upload e Ricerca) con breve descrizione, utili per l'orientamento iniziale.
- Griglia delle ultime immagini caricate (fino a 15, su 3 colonne), aggiornata automaticamente a ogni ritorno sulla Home; ogni anteprima è una miniatura quadrata 180×180 px con angoli arrotondati.
- Sidebar con lo spazio di archiviazione occupato (progress bar rispetto a TOTAL_STORAGE_GB) e i contatori di immagini, video e dimensione del database.
- Area grafici Matplotlib per l'analisi (tag più usati, andamento dei caricamenti).

2.2 Caricamento (Upload)

Scopo. Caricare file immagine e video tramite finestra di sistema o trascinamento (drag & drop), assegnare tag personalizzati e avviare la categorizzazione AI.

Modalità d'uso:

- Selezionare i file trascinandoli nell'area tratteggiata oppure cliccandola per aprire il dialog di sistema (è supportata la selezione multipla).
- Ogni file valido viene aggiunto alla Coda di Caricamento con stato iniziale “In attesa”. Il luogo non va indicato manualmente: viene ricavato dai metadati GPS del file (reverse geocoding) quando disponibili.
- Per assegnare tag mirati: cliccare la riga del file (il bordo diventa blu), digitare i tag e premere “🏷 Applica Tag ai file selezionati”. I tag scritti nella casella al momento del salvataggio, in assenza di selezione, vengono invece applicati a tutti i file come “tag di base”.
- Rimuovere un file dalla coda con la ✕ accanto alla riga, o svuotare l'intera coda con “Svuota”.
- Premere “Salva e Analizza (N)”: i file vengono elaborati uno alla volta in background. Lo stato di ogni riga si aggiorna in tempo reale (in attesa → elaborazione → completato / già presente / errore) e una barra globale mostra il progresso complessivo.

Un file con estensione non supportata, oltre la dimensione massima (200 MB) o già presente in coda viene scartato singolarmente al momento dell'aggiunta, con un avviso riepilogativo, senza bloccare l'inserimento degli altri file validi.

2.3 Ricerca

Scopo. Cercare immagini e video combinando una query testuale libera, filtri relazionali e ricerca semantica/visiva, con visualizzazione dei risultati in tempo reale.

Modalità d'uso:

1. Digitare una parola chiave o una descrizione libera nel campo di ricerca.
2. Scegliere il tipo di file (Tutto / Immagini / Video), un intervallo di date (Da / A) e/o un luogo.
3. In alternativa al testo, caricare un'immagine di riferimento per una ricerca visiva per similarità (reverse image search). Testo e immagine reference sono mutuamente esclusivi.
4. Premere “Avvia Ricerca”: i risultati compaiono come griglia di card, ciascuna con anteprima, metadati e un badge di accuratezza.
5. “Apri” apre il file (o la cartella che lo contiene) in Esplora File; il cestino 🗑 elimina il file in modo permanente (uploads + DuckDB + ChromaDB) previa conferma esplicita.

3. Progettazione del database

L'architettura dati di DeepSight adotta un approccio dual-database (ibrido) orchestrato dalla classe Database (implementata come Singleton). Poiché nessuno dei due strumenti da solo copre entrambe le esigenze, il sistema unifica l'interazione con due motori embedded locali complementari:

- **DuckDB (relazionale).** Salva i metadati strutturati in tabelle SQL classiche e consente query complesse con JOIN, filtri LIKE e aggregazioni rapide per i grafici della dashboard.
- **ChromaDB (vettoriale).** Memorizza gli embedding a 512 dimensioni generati da CLIP, indicizzati con algoritmo HNSW e distanza coseno, per ricerche ultra-rapide per similarità visiva o semantica.

I due database sono collegati logicamente da un ID numerico univoco generato da una sequenza DuckDB e riutilizzato come chiave/metadato in ChromaDB, compensando così i limiti funzionali di ciascuno strumento: DuckDB non offre ricerca vettoriale ad alte prestazioni, ChromaDB non supporta query relazionali complesse.

Tabella file_multimediali (DuckDB)

Campo	Tipo	Note
id	INTEGER	Chiave primaria, autogenerata da sequenza (nextval)
tipo	VARCHAR	'immagine' oppure 'video'
percorso_file	VARCHAR	UNIQUE, path assoluto sul disco
nome_file	VARCHAR	Nome del file
luogo	VARCHAR	Default 'Sconosciuto' — da reverse geocoding GPS
gps	VARCHAR	Default NULL, coordinate nel formato 'lat, lon'
data_creazione	VARCHAR	Default 'Sconosciuta' — data di scatto da EXIF
data_inserimento	VARCHAR	Data/ora del caricamento (gg/mm/aaaa HH:MM)
testo_ocr	VARCHAR	Testo estratto via OCR
num_facce	INTEGER	Conteggio volti rilevati (MTCNN)
dimensione_mb	DOUBLE	Spazio occupato sul disco in MB

Tabella 1- Schema di file_multimediali

Tabella tag_assegnati (DuckDB)

Campo	Tipo	Note
id_file	INTEGER	Chiave esterna verso file_multimediali.id
tag	VARCHAR	Etichetta testuale, convertita in minuscolo

Tabella 2 - tag_assegnati

Chiave primaria composta (id_file, tag) per evitare duplicati. La tabella tag è uno strumento aggiuntivo per efficientare la ricerca tramite query e JOIN, riducendo tempi di elaborazione e possibili errori.

Collezione ChromaDB (multimedia_clip)

La collezione vettoriale è configurata con metrica coseno (hnsw:space = cosine). Ogni vettore è identificato dalla stringa “{id_file}_frame_{idx}” e accompagnato da metadati che permettono di ricollegarlo al record relazionale e — per i video — allo specifico fotogramma:

Elemento	Contenuto
id	"{id_file}_frame_{idx}" — es. "12_frame_28"
embedding	Vettore CLIP a 512 dimensioni
metadata	file_id, nome_file, percorso, tipo, frame_idx, timestamp_sec, is_video

Tabella 3 – struttura file ChromaDB

3.1 Schema della Logica di Funzionamento

La logica complessiva della gestione dei dati si divide nei due flussi principali eseguiti dall'applicazione: il Flusso di Salvataggio (Upload) e il Flusso di Ricerca Avanzata.

Pipeline di Upload e Processing dei Dati

```
[Utente fa Drop/Seleziona File] -> [Controllo Duplicati SQL]
         |
    (Nuovo File) ──> [Copia fisica del file in Storage]
         |
         ▼
[Avvio Pipeline AI in Background Thread]
  ├── Estrazione EXIF (Data Scatto, GPS)
  ├── Esecuzione OCR (EasyOCR -> Testo)
  ├── Face Detection (MTCNN -> Conteggio e volti)
  └── Modello CLIP -> Generazione Embedding Vettoriale
         |
         ▼
[Invocazione insert_multimedia()]
  ├── 1. Generazione ID su DuckDB ed esecuzione INSERT
  ├── 2. Ciclo di INSERT nella tabella tag_assegnati
  └── 3. Esecuzione .add() su ChromaDB per l'indicizzazione semantica
```

Algoritmo di Ricerca Avanzata (Ibrido)

```
[Utente imposta i parametri e clicca Cerca]
                    |
       C'è una query testuale o immagine?
            ├── SÌ: Generazione Embedding tramite CLIP ──> Interrogazione ChromaDB
            |                                                      |
            |                                                      ▼
            |                                              Ottiene "ID_Trovati"
            |                                                      |
            |                                                      ▼
            |                                    Esegue cerca_per_filtri(ids_vettoriali)
            └── NO: Chiamata diretta cerca_per_filtri(ids_vettoriali=None)
                    |
                    ▼
       La query SQL ha prodotto record?
            ├── SÌ: Invia i record formattati alla UI
            └── NO: Fallback automatico su cerca_per_testo_tag() via LIKE SQL
```

4. Requisiti di Sistema

Questa sezione raccoglie l'insieme delle caratteristiche tecniche e operative necessarie allo sviluppo e all'esecuzione dell'applicativo DeepSight.

4.1 Piattaforma di sviluppo

Linguaggio e runtime

- **Python 3.12** (supportato 3.9–3.12), scelto per la maturità dell'ecosistema AI/ML e la rapidità di sviluppo di interfacce desktop.

Interfaccia grafica

- **CustomTkinter** — estensione moderna di Tkinter, usata per l'intera UI desktop.
- **tkinterdnd2** — integrata opzionalmente (con caricamento protetto da try/except) per il supporto drag & drop.

Persistenza dati

- **DuckDB** — motore relazionale embedded per i metadati strutturati.
- **ChromaDB (PersistentClient)** — database vettoriale embedded per gli embedding CLIP (HNSW, spazio coseno).

Librerie AI/ML e supporto

- **PyTorch** — framework di deep learning sottostante a tutti i modelli.
- **Transformers (HuggingFace)** — modello CLIP (openai/clip-vit-base-patch32) per embedding condivisi immagine/testo.
- **EasyOCR** — riconoscimento ottico dei caratteri multilingua (IT/EN).
- **facenet-pytorch (MTCNN + InceptionResnetV1)** — rilevamento e conteggio volti ed estrazione di embedding facciali.
- **OpenCV (cv2)** — apertura e campionamento dei fotogrammi video.
- **Pillow (PIL)** — caricamento immagini ed estrazione dei metadati EXIF (data, GPS).
- **Matplotlib (FigureCanvasTkAgg)** — grafici della dashboard integrati nei frame CustomTkinter.
- **deep-translator** — traduzione IT→EN delle query testuali (CLIP rende meglio in inglese).
- **requests + Nominatim (OpenStreetMap)** — reverse/forward geocoding, gratuito e senza API key.

Ambiente di esecuzione e configurazione

Applicazione desktop standalone, eseguibile localmente senza connessione a servizi cloud: tutti i modelli AI girano in locale per riservatezza. Uno script di avvio dedicato (start_app.py) verifica automaticamente la presenza di tutte le dipendenze critiche prima di lanciare l'interfaccia, segnalando eventuali pacchetti mancanti con il comando pip/conda corrispondente. Un file di configurazione centralizzato (Config_Start/config.py) raccoglie tutti i percorsi e i parametri dei modelli, evitando hard-coding nei singoli moduli. Ogni dipendenza AI è isolata dietro import “lazy” per non appesantire l'avvio.

4.2 Architettura software

DeepSight adotta un'architettura modulare a livelli (layered), con una netta separazione tra interfaccia utente, logica applicativa/AI e persistenza dati, tenuti insieme da un modulo di configurazione condiviso.

Modulo	Responsabilità
Config_Start/config.py	Configurazione centralizzata (percorsi, parametri AI, formati supportati)
UI/home.py	Dashboard principale, statistiche, grafici, drag & drop a livello finestra
UI/upload.py	Caricamento file, validazione, coda multi-file, avvio pipeline AI in background
UI/ricerca.py	Interfaccia e logica di ricerca avanzata (filtri, similarità, fallback testuale)
AI_Model/AI_engine.py	Motore AI: embedding CLIP, OCR, EXIF, rilevamento/embedding facciale, campionamento video
Database/database.py	Livello di persistenza: interfaccia unificata su DuckDB e ChromaDB
Config_Start/start_app.py	Bootstrap dell'applicazione e verifica dipendenze

Tabella 4 - Moduli principali

Modelli di intelligenza artificiale utilizzati

Modello	Funzione	Note
CLIP	Embedding condiviso immagine/testo (512-d) per ricerca semantica e per similarità	Lazy loading al primo utilizzo; caricamento offline preferito
EasyOCR	Estrazione testo da immagini (IT/EN)	GPU opzionale se disponibile
MTCNN + InceptionResnetV1	Rilevamento e conteggio volti, embedding facciale per ricerca per identità	GPU opzionale se disponibile

Tabella 5 - Modelli AI

Tutti i modelli sono caricati in lazy loading: rimangono a None finché l'utente non esegue il primo upload, così da garantire un avvio istantaneo dell'interfaccia.

4.3 Requisiti minimi hardware

Coerentemente con il requisito non funzionale del progetto (esecuzione su un normale PC anche senza GPU), l'applicazione è progettata per funzionare interamente su CPU, con supporto opzionale alla GPU per velocizzare OCR e riconoscimento facciale.

Componente	Minimo	Consigliato
CPU	Multi-core 64-bit (es. Intel Core i5)	CPU multi-core più recente/performante
RAM	8 GB	16 GB
Storage	≈109 GB (100 archivio + 5 DB + 4 cache modelli)	300 GB
GPU	Non richiesta (modalità CPU-only)	GPU NVIDIA con supporto CUDA
OS	Windows 10/11 (64-bit); anche macOS/Linux	—
Rete	Solo per il primo download dei pesi dei modelli	—

Tabella 6 - Requisiti hardware

5. Descrizione delle classi (OOP)

Il progetto è organizzato in quattro classi principali, una per ogni componente chiave (dati, finestra principale, upload, ricerca). Il motore AI (AI_engine.py) è invece implementato come modulo funzionale — non come classe — con i modelli tenuti in variabili globali caricate in lazy loading: essendo modelli unici e condivisi da tutta l'app, non era necessario incapsularli in un oggetto.

Database (in database.py)

Gestisce in modo unificato la persistenza su DuckDB e ChromaDB. Implementata come Singleton, in modo che tutta l'applicazione condivida sempre la stessa connessione.

Attributi principali	Metodi principali
conn (connessione DuckDB)	insert_multimedia() — salva metadati, tag ed embedding
chroma_client / chroma_collection	cerca_per_embedding() — ricerca per similarità
_instance (istanza singleton)	cerca_per_filtri() / cerca_per_testo_tag() / cerca_per_coordinate()
_inizializzato (guardia init)	get_statistiche() / elimina_file()

Tabella 7 – classe databas

DeepSightApp (in home.py)

Classe principale dell'applicazione (eredita da ctk.CTk, con mixin tkinterdnd2 quando disponibile); rappresenta la finestra e funge da controller centrale per la navigazione.

Attributi principali	Metodi principali
navbar, frame_home / upload / ricerca	mostra_pagina() — cambia schermata attiva
filtro_tipo (stato condiviso dei filtri)	aggiorna_statistiche_storage() — spazio occupato
_responsive_fonts, color_* (tema)	aggiorna_ultime_immagini() / aggiorna_grafici()

Tabella 8 - classe DeepSightApp

UploadFrame (in upload.py)

Frame CustomTkinter per il caricamento di nuovi contenuti; coordina copia file, analisi AI e salvataggio, gestendo una coda multi-file.

Attributi principali	Metodi principali
coda (lista elementi in attesa)	seleziona_file() / gestisci_drop() — input file
_elaborazione_in_corso	avvia_pipeline_upload() — avvia il thread di elaborazione
drop_zone, tag_entry, progress bar	_processa_file_singolo() / _on_coda_completata()

Tabella 9 - classe uploadframe

RicercaFrame (in ricerca.py)

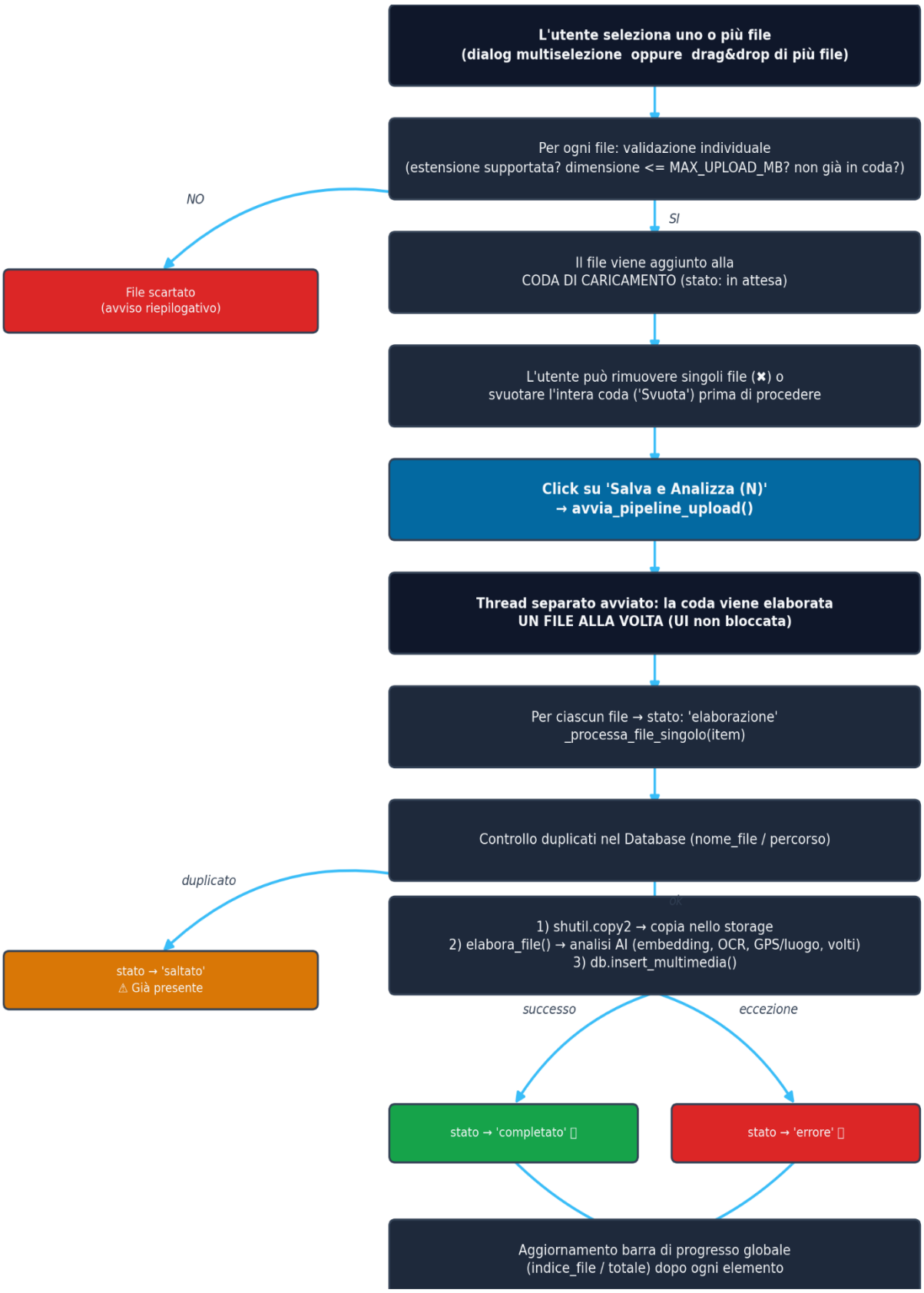
Frame CustomTkinter per la ricerca avanzata; traduce i filtri impostati dall'utente in query verso la classe Database e calibra i punteggi di accuratezza.

Attributi principali	Metodi principali
ricerca_img_reference (img. di query)	avvia_ricerca() / _esegui_ricerca_reale()
_thumbnail_refs (riferimenti img. UI)	_mostra_risultati() / _crea_card_risultato()
_dialog_reference_aperto (guardia)	seleziona_reference() / azzera_filtri() / _conferma_elimina()

Tabella 10 - classe ricercaframe

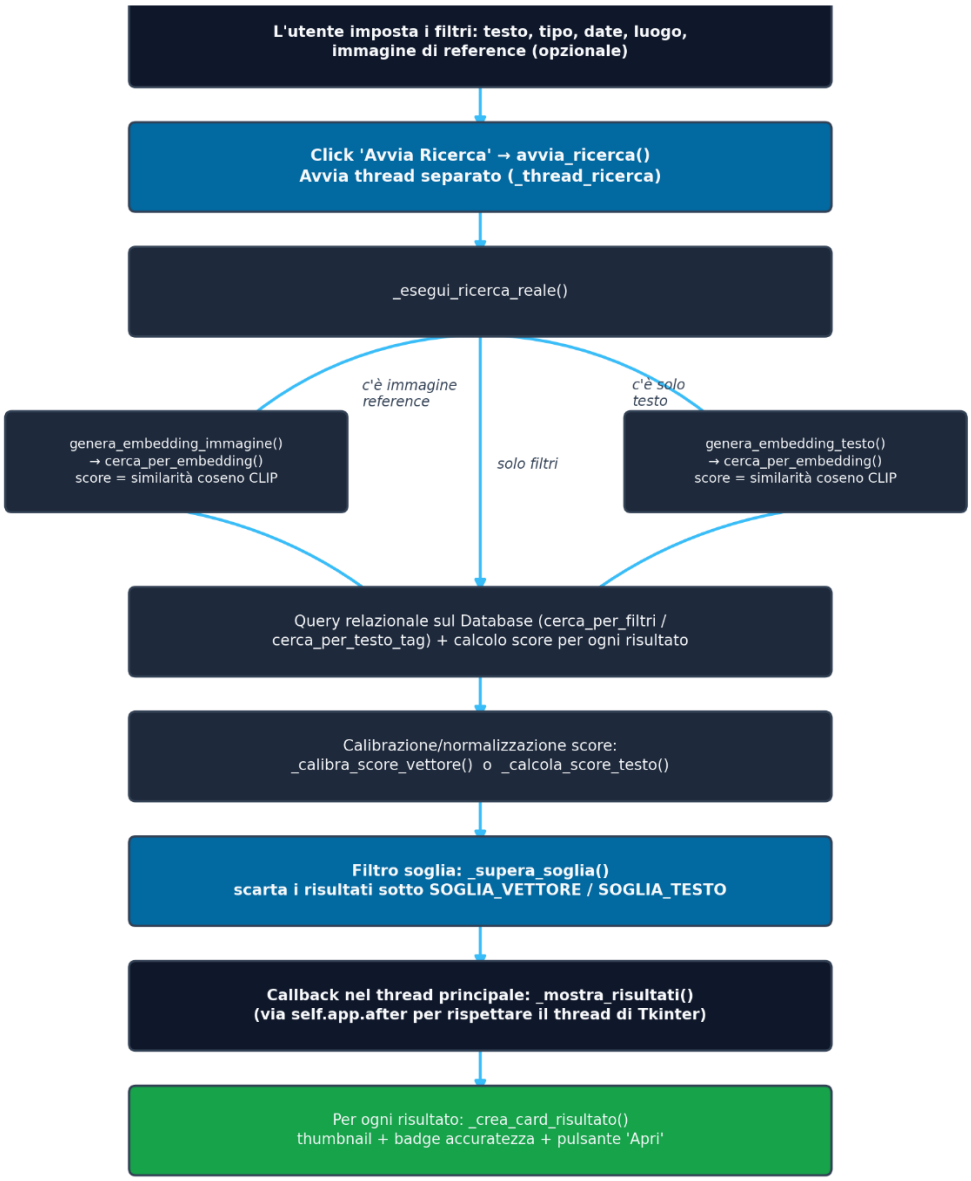
6. Flussi di funzionamento

6.1 Pipeline di Upload e processing



1. L'utente seleziona o trascina uno o più file nell'area di upload.
2. Vengono validati estensione, dimensione massima (200 MB) e assenza di duplicati in coda; i file validi vengono accodati.
3. Alla pressione di “Salva e Analizza”, un thread separato elabora la coda un file alla volta (per non bloccare il ciclo Tkinter).
4. Per ogni file: controllo duplicati sul DB (solo sul record, non sulla presenza fisica); copia nello storage; analisi AI — embedding CLIP, OCR (EasyOCR), metadata EXIF (data, GPS→luogo), rilevamento volti (MTCNN).
5. Per i video il campionamento è adattivo: 30 frame distribuiti uniformemente (fallback 1 frame/sec sotto i 30 s); ogni frame è indicizzato singolarmente. I frame temporanei vengono rimossi al termine.
6. insert_multimedia salva i metadati relazionali su DuckDB e gli embedding su ChromaDB nella stessa operazione logica, legandoli con lo stesso ID.
7. Stato e completamento sono comunicati alla UI in modo thread-safe tramite after(); al termine la dashboard si aggiorna automaticamente.

6.2 Ricerca avanzata (ibrida)



1. L'utente imposta i criteri (testo o immagine di query, tipo, luogo, intervallo date).
2. Se è prevista ricerca semantica/per similarità, viene generato un embedding di query (testo o immagine via CLIP) e interrogato ChromaDB (cerca_per_embedding), che restituisce gli ID più simili con score.
3. Gli ID ottenuti vengono passati a DuckDB (cerca_per_filtri), che applica i filtri tradizionali; il luogo digitato viene convertito in coordinate (forward geocoding) e usato per intersecare i risultati per prossimità (cerca_per_coordinate).
4. Se la ricerca vettoriale non produce risultati, il sistema ricade automaticamente sulla ricerca testuale (cerca_per_testo_tag) su nome file, tag e testo OCR, rispettando i filtri di tipo e luogo.
5. Gli score vengono calibrati (funzioni distinte per immagine vs. testo→immagine) e filtrati per soglia; i risultati aggregati (metadati + tag) sono restituiti alla UI per la visualizzazione, l'apertura o l'eliminazione.

7. Deployment e installazione

Tipo di distribuzione. Applicazione desktop standalone monoutente, eseguita interamente in locale su PC Windows (supportati anche macOS/Linux). Non è previsto alcun deployment server/cloud: UI, logica, modelli AI e database risiedono sulla stessa macchina.

Installazione (Windows)

1. Copiare la cartella del progetto (o clonare il repository) sul PC di destinazione.
2. Eseguire `install.bat`: lo script individua o scarica automaticamente Python 3.12 (installazione solo-utente, senza permessi di amministratore), verifica la presenza di tkinter, crea l'ambiente virtuale `.venv` e installa le dipendenze da requirements.txt.
3. Avviare l'app con `DeepSight.bat` — unico avvio supportato su Windows: attiva l'ambiente virtuale, imposta DEEPSIGHT_LAUNCHER=1 ed esegue start_app.py, che verifica le dipendenze prima di lanciare l'interfaccia.

Installazione (macOS / Linux)

1. Rendere eseguibile ed eseguire `./install.sh`: individua un interprete Python 3.9–3.12, verifica tkinter (pacchetto di sistema, es. python3-tk), crea .venv e installa le dipendenze.
2. Avviare con source .venv/bin/activate && python Config_Start/start_app.py.

Configurazione e primo avvio

Configurazione. Tutti i parametri modificabili (percorsi di storage, modello CLIP, lingue OCR, limite upload) sono raccolti in Config_Start/config.py. Le cartelle necessarie vengono create automaticamente al primo avvio.

Primo avvio. Al primo upload effettivo l'app scarica una tantum i pesi pre-addestrati (CLIP da HuggingFace, EasyOCR, FaceNet) e li mette in cache: la connessione internet serve solo in questa fase. Negli utilizzi successivi l'app funziona completamente offline. L'app rileva automaticamente CUDA e la usa per accelerare OCR e riconoscimento facciale, altrimenti opera in modalità CPU-only.

Nota operativa. Copiare o trascinare un file direttamente nella cartella uploads/ NON lo indicizza: va sempre usata la UI di Upload (drag & drop o selezione), che esegue la pipeline AI. La ricerca per luogo trova solo le foto con coordinate GPS nei metadati EXIF (tipicamente scatti da smartphone/camera); screenshot e immagini scaricate dal web restano “Sconosciuto”.

Vincoli sulle versioni delle dipendenze

- `torch==2.2.2` e `torchvision==0.17.2` — richiesti da facenet-pytorch (wheel precompilate anche per Python 3.12).
- `transformers==4.39.3` (<4.40) — le versioni successive disabilitano il backend PyTorch con torch < 2.4, rompendo il caricamento di CLIP.
- `numpy<2` — compatibilità con questa versione di torch/transformers.
- `chromadb>=1.0.0` — dalla 1.0 (core in Rust) non dipende più da chroma-hnswlib, evitando la necessità dei Microsoft C++ Build Tools su Windows/Python 3.12.

8. Documentazione delle API (funzioni e metodi)

Questa sezione documenta, modulo per modulo, ogni funzione e metodo pubblico e privato con firma, scopo, parametri accettati e valore di ritorno, sullo stile di una API reference. Per convenzione, i nomi che iniziano con underscore (_) indicano funzioni di uso interno al modulo.

8.1 config.py — Configurazione centralizzata

Modulo di sole costanti importato da tutti gli altri componenti. All'import calcola i percorsi assoluti e crea le cartelle mancanti. Non espone funzioni: espone variabili globali condivise.

Costante	Valore / significato
BASE_DIR	Radice del progetto (derivata dal percorso del file)
UPLOAD_DIR / IMG_DIR / VIDEO_DIR	Cartelle di storage: uploads, uploads/immagini, uploads/video
DB_PATH	File DuckDB: Database/deepsight.db
CHROMA_DIR	Cartella persistente di ChromaDB: Database/chroma_vectors
CLIP_MODEL_NAME	"openai/clip-vit-base-patch32"
OCR_LANGUAGES	['it', 'en']
CLIP_EMBEDDING_DIM	512 (dimensione embedding CLIP)
TOTAL_STORAGE_GB	500.0 (capacità di riferimento per la barra storage)
MAX_UPLOAD_MB	200 (dimensione massima per singolo file)
ESTENSIONI_IMMAGINI	.png .jpg .jpeg .webp .bmp .gif
ESTENSIONI_VIDEO	.mp4 .avi .mkv .mov .flv .wmv

8.2 start_app.py — Bootstrap e verifica dipendenze

Metodo: _is_conda_env

Firma: def _is_conda_env()
Descrizione: Rileva se l'interprete corrente è in un ambiente conda (variabile CONDA_DEFAULT_ENV o cartella conda-meta).
Parametri: —
Ritorno: bool

Metodo: controlla_dipendenze

Firma: controlla_dipendenze()
Descrizione: Verifica che tutte le librerie esterne critiche siano installate e importabili; in caso di pacchetti mancanti o incompatibilità di versione (es. NumPy 1.x vs 2.x) stampa il comando pip/conda suggerito e interrompe l'avvio.
Parametri: —
Ritorno: None; termina con sys.exit(1) in caso di errori

Metodo: _verifica_avvio_da_launcher

Firma: _verifica_avvio_da_launcher()
Descrizione: Su Windows impone l'avvio esclusivo tramite DeepSight.bat (verifica DEEPSIGHT_LAUNCHER == "1"); su macOS/Linux l'avvio diretto resta consentito.
Parametri: —
Ritorno: None; termina con sys.exit(1) se avviato fuori dal launcher su Windows

8.3 database.py — Classe Database

class Database — Livello di persistenza unificato su DuckDB e ChromaDB, implementato come Singleton tramite __new__.

Metodo: __new__

Firma: def __new__(cls)
Descrizione: Implementa il pattern Singleton: crea l'istanza una sola volta e la riusa a ogni successiva chiamata.
Parametri: —
Ritorno: L'unica istanza di Database

Metodo: __init__

Firma: def __init__(self)
Descrizione: Inizializza (una sola volta, grazie a _inizializzato) la connessione DuckDB, crea le tabelle e apre/crea la collezione ChromaDB con metrica coseno.
Parametri: —
Ritorno: None

Metodo: _crea_tabelle_duckdb

Firma: def _crea_tabelle_duckdb(self)
Descrizione: Crea la sequenza ID e le tabelle file_multimediali e tag_assegnati se non esistono; esegue un ALTER TABLE di sicurezza per aggiungere la colonna gps ai DB creati con versioni precedenti.
Parametri: —
Ritorno: None

Metodo: insert_multimedia

Firma: def insert_multimedia(self, tipo, percorso_file, tag_list=None, luogo='Sconosciuto', gps=None, data_creazione='Sconosciuta', embedding_clip=None, frame_embeddings=None, testo_ocr='', num_facce=0)
Descrizione: Salva un file: (1) INSERT dei metadati su DuckDB con RETURNING id; (2) inserimento dei tag in tag_assegnati legati all'ID; (3) salvataggio dell'embedding singolo (immagini/primo frame) e/o di frame_embeddings (video) su ChromaDB con lo stesso ID come chiave. Controlla i duplicati sul percorso.
Parametri: tipo (str), percorso_file (str), tag_list (list[str]), luogo (str), gps (str), data_creazione (str), embedding_clip (list[float]), frame_embeddings (list[dict]), testo_ocr (str), num_facce (int)
Ritorno: int — ID assegnato; None se duplicato o errore

Metodo: cerca_per_embedding

Firma: def cerca_per_embedding(self, embedding_query, top_n=20)
Descrizione: Interroga ChromaDB confrontando l'embedding di query con quelli salvati (distanza coseno). Per i video raggruppa per file_id e maintains solo il frame con score più alto, restituendo frame_idx e timestamp_sec associati.
Parametri: embedding_query (list[float], 512), top_n (int)
Ritorno: list[dict] ordinata per score decrescente (id, percorso_file, nome_file, tipo, score, is_video, frame_idx, timestamp_sec)

Metodo: cerca_per_filtri

Firma: def cerca_per_filtri (self, tipo=None, data_inizio=None, data_fine=None, luogo=None, ids_vettoriali=None)
Descrizione: Costruisce ed esegue una query SQL dinamica su DuckDB. Se ids_vettoriali è presente, aggiunge una clausola IN (...) per restringere i risultati a quelli trovati dalla ricerca vettoriale; combina i filtri tradizionali e aggrega i tag con STRING_AGG.
Parametri: tipo (str), data_inizio (str), data_fine (str), luogo (str), ids_vettoriali (list[int])
Ritorno: list[dict] con tutti i metadati del media (max 50)

Metodo: cerca_per_testo_tag

Firma: def cerca_per_testo_tag(self, testo, tipo=None, luogo=None)
Descrizione: Ricerca fallback tradizionale via SQL LIKE su nome file, testo OCR e tag. I parametri opzionali tipo e luogo aggiungono i rispettivi filtri, permettendo di combinare contenuto e località.
Parametri: testo (str), tipo (str), luogo (str)
Ritorno: list[dict] con i metadati dei file corrispondenti (max 50)

Metodo: cerca_per_coordinate

Firma: def cerca_per_coordinate(self, lat, lon, raggio_lat=1.0, raggio_lon=1.0, tipo=None)
Descrizione: Ricerca di prossimità geografica: filtra la colonna gps per bounding box lat±raggio_lat, lon±raggio_lon attorno al punto di query (tipicamente ottenuto dal forward geocoding del luogo digitato).
Parametri: lat (float), lon (float), raggio_lat (float), raggio_lon (float), tipo (str)
Ritorno: list[dict] con i file nelle vicinanze (max 200)

Metodo: get_statistiche

Firma: def get_statistiche(self)
Descrizione: Aggrega le metriche per la dashboard con 3 query: conteggio e peso totale di immagini/video, i 10 tag più usati e l'andamento dei caricamenti per giorno.
Parametri: —
Ritorno: dict {count_img, count_vid, size_totale_mb, tag_frequenze, caricamenti_per_giorno}

Metodo: get_percorso_da_id

Firma: def get_percorso_da_id(self, id_file)
Descrizione: Restituisce il percorso del file dato il suo ID.
Parametri: id_file (int)
Ritorno: str | None

Metodo: elimina_file

Firma: def elimina_file(self, id_file)
Descrizione: Elimina definitivamente un file: rimuove il file fisico da uploads, cancella record e tag da DuckDB ed elimina tutti gli embedding associati da ChromaDB (tutti i frame, per i video). Operazione irreversibile.
Parametri: id_file (int)
Ritorno: tuple (ok: bool, errore: str | None)

Metodo: chiudi_connessione

Firma: def chiudi_connessione(self)
Descrizione: Chiude la connessione a DuckDB.
Parametri: —
Ritorno: None

8.4 AI_engine.py — Motore AI (modulo funzionale)

Fornisce embedding CLIP (immagini e testo), OCR, estrazione metadata EXIF, reverse/forward geocoding, riconoscimento facciale ed elaborazione video. I modelli sono tenuti in variabili globali private (_clip_model, _clip_processor, _ocr_reader, _mtcnn, _facenet) e caricati solo al primo utilizzo.

Metodo: _carica_clip

Firma: def _carica_clip()
Descrizione: Carica CLIP (modello + processor) una sola volta. Tenta prima il caricamento offline (local_files_only=True) per evitare timeout di rete; ricade sul download solo al primo utilizzo.
Parametri: —
Ritorno: None (popola le globali)

Metodo: _carica_ocr

Firma: def _carica_ocr()
Descrizione: Carica EasyOCR (lingue da OCR_LANGUAGES) una sola volta, usando la GPU se disponibile.
Parametri: —
Ritorno: None

Metodo: _carica_facenet

Firma: def _carica_facenet()
Descrizione: Carica MTCNN e InceptionResnetV1 (pre-addestrato su VGGFace2) una sola volta.
Parametri: —
Ritorno: None

Metodo: _extract_tensor

Firma: def _extract_tensor(output)
Descrizione: Normalizza l'output di get_image_features / get_text_features tra versioni diverse di transformers: estrae .pooler_output se presente (BaseModelOutputWithPooling), altrimenti usa il tensore grezzo.
Parametri: output (Tensor | BaseModelOutputWithPooling)
Ritorno: torch.Tensor

Metodo: genera_embedding_immagine

Firma: def genera_embedding_immagine(percorso_img)
Descrizione: Genera l'embedding CLIP visivo (512-d) normalizzato di un'immagine, per la ricerca per similarità.
Parametri: percorso_img (str)
Ritorno: list[float] (512) | None

Metodo: genera_embedding_testo

Firma: def genera_embedding_testo(testo)
Descrizione: Genera l'embedding CLIP di una query testuale. Corregge alcune ambiguità italiane note (es. “macchina”→“car”), traduce in inglese, applica il prompt ensembling (media di 5 prompt contestualizzati) e normalizza il vettore.
Parametri: testo (str)
Ritorno: list[float] (512) | None

Metodo: genera_embedding_facciale

Firma: def genera_embedding_facciale(percorso_img)
Descrizione: Rileva le facce e restituisce l'embedding (512-d) della prima faccia trovata, per la ricerca per identità.
Parametri: percorso_img (str)
Ritorno: list[float] (512) | None

Metodo: estrai_testo_ocr

Firma: def estrai_testo_ocr(percorso_img)
Descrizione: Esegue l'OCR sull'immagine con EasyOCR e unisce il testo riconosciuto in un'unica stringa.
Parametri: percorso_img (str)
Ritorno: str (eventualmente vuota)

Metodo: _converti_gps

Firma: def _converti_gps(gps_info)
Descrizione: Converte i dati GPS EXIF in coordinate decimali forzando float() su ogni componente IFDRational (fix di un bug che rompeva il geocoding).
Parametri: gps_info (dict EXIF)
Ritorno: tuple (lat, lon) | None

Metodo: riconosci_luogo_da_gps

Firma: def riconosci_luogo_da_gps(lat, lon)
Descrizione: Reverse geocoding via Nominatim: converte coordinate in un nome di luogo leggibile. Usa una cache in memoria per evitare chiamate ripetute.
Parametri: lat (float), lon (float)
Ritorno: str | None

Metodo: riconosci_coordinate_da_testo

Firma: def riconosci_coordinate_da_testo(testo)
Descrizione: Forward geocoding via Nominatim: converte un nome di luogo in coordinate decimali. Con cache in memoria.
Parametri: testo (str)
Ritorno: tuple (lat, lon) | None

Metodo: estrai_metadata_exif

Firma: def estrai_metadata_exif(percorso_img)
Descrizione: Estrae data di scatto e GPS dai metadati EXIF; se presenti coordinate, tenta il reverse geocoding per ricavare il nome del luogo (fallback “GPS: lat, lon”).
Parametri: percorso_img (str)
Ritorno: dict {data, luogo, gps}

Metodo: elabora_immagine

Firma: def elabora_immagine(percorso_file)
Descrizione: Analisi completa di un'immagine: embedding CLIP, OCR, metadata EXIF (data, GPS, luogo) e conteggio volti.
Parametri: percorso_file (str)
Ritorno: dict con tutte le informazioni | None

Metodo: elabora_video

Firma: def elabora_video(percorso_file, callback_progresso=None)
Descrizione: Campiona esattamente TARGET_FRAMES=30 frame distribuiti uniformemente (intervallo = durata/30; fallback 1 frame/sec sotto i 30 s) e, per ogni frame, genera embedding CLIP e OCR; rimuove i frame temporanei al termine.
Parametri: percorso_file (str), callback_progresso (callable)
Ritorno: list[dict] (uno per frame: embedding, timestamp_sec, frame_idx) | None

Metodo: elabora_file

Firma: def elabora_file(percorso_file, callback_progresso=None)
Descrizione: Entry point del modulo AI: in base all'estensione delega a elabora_immagine o elabora_video. Il callback opzionale aggiorna la UI sullo stato.
Parametri: percorso_file (str), callback_progresso (callable)
Ritorno: dict (immagini) o list[dict] (video) | None

8.5 home.py — Classe DeepSightApp

Metodo: __init__

Firma: def __init__(self)
Descrizione: Costruisce l'applicazione principale: inizializza il tema grafico, la finestra principale, gestisce la configurazione dei percorsi di sistema, la barra di navigazione (navbar), istanzia le tre schermate operative primarie dell'applicazione e mostra la schermata Home di default.
Parametri: —
Ritorno: None

Metodo: trigger_re_layout

Firma: def trigger_re_layout(self)
Descrizione: Propaga in modo centralizzato a tutte le sotto-pagine aperte il segnale di ricalcolo dinamico del layout responsive, basandosi sulle dimensioni in larghezza correnti rilevate a livello di finestra.
Parametri: —
Ritorno: None

Metodo: _font

Firma: def _font(self, size, weight='normal', family='Inter')
Descrizione: Genera e registra un'istanza di CTkFont all'interno del sistema interno di tracciamento. Questo permette la scalabilità automatica in base al ridimensionamento della finestra principale.
Parametri: size (int); weight (str); family (str)
Ritorno: CTkFont

Metodo: _on_resize_root

Firma: def _on_resize_root(self, event)
Descrizione: Intercetta in tempo reale l'evento nativo di ridimensionamento della finestra root e pianifica l'adeguamento proporzionale delle dimensioni dei caratteri testuali applicando una logica di debounce per ottimizzare le prestazioni.
Parametri: event (tk Event)
Ritorno: None

Metodo: _applica_scala_font

Firma: def _applica_scala_font(self, larghezza, altezza)
Descrizione: Calcola il coefficiente di riscalaggio rispetto alla risoluzione standard di riferimento (1200×720). Applica una protezione (clamp) limitando il fattore tra 0.85 e 1.4, aggiornando di conseguenza tutti i font e le icone responsive dell'interfaccia.
Parametri: larghezza (int); altezza (int)
Ritorno: None

Metodo: mostra_pagina

Firma: def mostra_pagina(self, nome_pagina)
Descrizione: Nasconde l'albero visivo di tutte le schermate registrate e rende visibile esclusivamente la pagina passata come argomento; aggiorna graficamente lo stato del pulsante di selezione nella navbar.
Parametri: nome_pagina (str): 'home' | 'upload' | 'ricerca'
Ritorno: None

Metodo: crea_pagina_home

Firma: def crea_pagina_home(self)
Descrizione: Configura ed assembla l'infrastruttura completa dei widget costituenti la dashboard di benvenuto: include le card operative di accesso rapido, la griglia flessibile a scorrimento dedicata alle ultime immagini rilevate, la sidebar laterale di stato dello storage fisico e i pannelli per i grafici statistici.
Parametri: —
Ritorno: None

Metodo: re_layout_home

Firma: def re_layout_home(self, is_mobile, width)
Descrizione: Riorganizza la disposizione e il flusso spaziale degli elementi della schermata Home adattandoli in funzione della larghezza totale disponibile o della visualizzazione ottimizzata in modalità mobile.
Parametri: is_mobile (bool); width (int)
Ritorno: None

Metodo: immagine

Firma: def immagine(self, pil_img)
Descrizione: Accetta un oggetto immagine basato su libreria PIL e ne esegue la conversione standardizzata nel formato colore RGBA per uniformare la pipeline di visualizzazione grafica.
Parametri: pil_img (PIL.Image)
Ritorno: PIL.Image (RGBA)

Metodo: get_dimensione_cartella_mb

Firma: def get_dimensione_cartella_mb(self, cartella)
Descrizione: Esegue una scansione dello spazio orizzontale occupato calcolando il peso complessivo espresso in MegaByte (MB) dei soli file posizionati direttamente all'interno del percorso indicato, escludendo l'analisi ricorsiva delle sottocartelle.
Parametri: cartella (str)
Ritorno: float (0 se la cartella risulta inesistente)

Metodo: aggiorna_statistiche_storage

Firma: def aggiorna_statistiche_storage(self)
Descrizione: Esegue il conteggio dei file di tipo immagine e video, determina l'esatto spazio occupato su disco e provvede ad aggiornare i widget contatori, le barre di avanzamento grafiche e la metrica di dimensione associata al file database.
Parametri: —
Ritorno: None

Metodo: aggiorna_ultime_immagini

Firma: def aggiorna_ultime_immagini(self)
Descrizione: Analizza i file risiedenti nel percorso IMG_DIR, effettuando un ordinamento basato sulla data dell'ultima modifica. Mostra all'interno della dashboard un massimo di 15 miniature con dimensioni 180×180 ed angoli smussati organizzate su 3 colonne; in assenza di file mostra un elemento segnaposto.
Parametri: —
Ritorno: None

8.6 upload.py — Classe UploadFrame

Metodo: __init__

Firma: def __init__(self, parent, app)
Descrizione: Inizializza ed assembla la schermata dedicata alla funzionalità di caricamento dei media. Integra la drop zone interattiva, l'area di inserimento per i tag testuali, i bottoni operativi di invio dati, la coda visiva con il relativo pulsante per svuotarla e aggancia i listener per la gestione del drag & drop.
Parametri: parent (widget); app (DeepSightApp)
Ritorno: None

Metodo: re_layout

Firma: def re_layout(self, is_mobile, width)
Descrizione: Ricalcola posizioni e allineamenti geometrici di tutti i moduli grafici interni della pagina di upload a seconda dello spazio in larghezza disponibile a schermo.
Parametri: is_mobile (bool); width (int)
Ritorno: None

Metodo: _rimuovi_placeholder/ _inserisci_placeholder

Firma: def _rimuovi_placeholder(self, event) | def _inserisci_placeholder(self, event)
Descrizione: Controllano in modo coordinato la visualizzazione e la rimozione del testo informativo di esempio posizionato all'interno del campo di inserimento Tag insieme con gli eventi di focus e blur.
Parametri: event (tk Event)
Ritorno: None

Metodo: seleziona_file

Firma: def seleziona_file(self)
Descrizione: Invoca la finestra di dialogo nativa del sistema operativo abilitata per la multiselezione di file (askopenfilenames) ed inserisce i percorsi validi all'interno della coda di gestione locale. L'operazione viene ignorata se un processo di analisi è già attivo.
Parametri: —
Ritorno: None

Metodo: gestisci_drop

Firma: def gestisci_drop(self, event)
Descrizione: Elabora l'evento scatenato dal rilascio di uno o più file trascinati sopra l'area target della drop zone, ripulendo e formattando la stringa dei percorsi fisici tramite la funzione tk.splitlist.
Parametri: event (evento DnD)
Ritorno: None

Metodo: _aggiungi_piu_files

Firma: def _aggiungi_piu_files(self, files)
Descrizione: Prende in carico una collezione di percorsi, avvia la validazione globale di ciascuno e ne pianifica l'inserimento in coda, generando a schermo un pannello riepilogativo per mostrare gli eventuali file scartati dal sistema.
Parametri: files (list[str])
Ritorno: None

Metodo: _aggiungi_alla_coda

Firma: def _aggiungi_alla_coda(self, percorso)
Descrizione: Verifica l'idoneità di un singolo file esaminando l'esistenza fisica su disco, il formato dell'estensione, il limite massimo di dimensione impostato a 200 MB ed escludendo tentativi di inserimento di file già presenti in coda. Se l'esito è positivo istanzia la struttura dell'elemento.
Parametri: percorso (str)
Ritorno: bool (True se l'elemento viene inserito con successo)

Metodo: _genera_thumbnail_coda

Firma: def _genera_thumbnail_coda(self, item)
Descrizione: Genera in modo dinamico un'anteprima visiva ridimensionata a 40×40 pixel sotto forma di CTkImage: estrae l'immagine originale oppure cattura il fotogramma iniziale del file video sfruttando le librerie di OpenCV.
Parametri: item (dict)
Ritorno: CTkImage | None (in caso di errore restituisce una emoji di fallback)

Metodo: _mostra_coda_vuota

Firma: def _mostra_coda_vuota(self)
Descrizione: Attiva e rende visibile l'elemento grafico segnaposto all'interno dell'interfaccia nel momento in cui la coda di lavoro risulta priva di elementi da visualizzare.
Parametri: —
Ritorno: None

Metodo: _crea_riga_coda

Firma: def _crea_riga_coda(self, item)
Descrizione: Costruisce l'architettura dei widget associati ad una riga dell'elenco in coda: visualizza l'anteprima miniatura, la denominazione del file, la stringa informativa di tipo/dimensione, i tag associati all'elemento specifico, lo stato corrente evidenziato cromaticamente e il tasto per la rimozione manuale. La riga risponde alla selezione utente.
Parametri: item (dict)
Ritorno: None

Metodo: _toggle_selezione

Firma: def _toggle_selezione(self, item)
Descrizione: Inverte lo stato di selezione logica e grafica associato ad una riga della coda, permettendo l'assegnazione mirata dei tag. La funzionalità viene inibita se è in corso un'operazione di elaborazione globale.
Parametri: item (dict)
Ritorno: None

Metodo: _aggiorna_evidenza_riga

Firma: def _aggiorna_evidenza_riga(self, item)
Descrizione: Modifica l'aspetto visivo ed il colore dei bordi perimetrali della riga specificata rispecchiando in tempo reale lo stato aggiornato di selezione.
Parametri: item (dict)
Ritorno: None

Metodo: _applica_tag_selezionati

Firma: def _applica_tag_selezionati(self)
Descrizione: Estrae il testo presente all'interno della casella dei tag globale e lo associa esclusivamente ai file contrassegnati come selezionati nella coda evitando l'inserimento di valori duplicati. Mostra un avviso se il campo è vuoto o se non sono stati scelti elementi.
Parametri: —
Ritorno: None

Metodo: _rimuovi_dalla_coda / _svuota_coda

Firma: def _rimuovi_dalla_coda(self, item) | def _svuota_coda(self)
Descrizione: Provvedono a disassociare ed eliminare dall'albero grafico un singolo elemento mirato oppure l'interezza degli oggetti presenti in coda, ripristinando il placeholder visivo di coda vuota. La funzione è bloccata durante le analisi.
Parametri: item (dict) / —
Ritorno: None

Metodo: _aggiorna_conteggio_coda

Firma: def _aggiorna_conteggio_coda(self)
Descrizione: Ricalcola ed aggiorna dinamicamente la stringa testuale dell'intestazione dell'elenco in coda e la label numerica presente sul pulsante principale di attivazione pipeline ('Salva e Analizza (N)').
Parametri: —
Ritorno: None

Metodo: _aggiorna_testo_drop

Firma: def _aggiorna_testo_drop(self, n)
Descrizione: Modifica in tempo reale il messaggio testuale esposto all'interno della drop zone per notificare formalmente all'utente il quantitativo esatto di file inseriti nel sistema.
Parametri: n (int)
Ritorno: None

Metodo: avvia_pipeline_upload

Firma: def avvia_pipeline_upload(self)
Descrizione: Punto di ingresso per il motore di importazione: se la coda contiene elementi validi, istanzia e lancia un thread asincrono separato delegato all'elaborazione in sequenza sequenziale di ciascun file presente in lista.
Parametri: —
Ritorno: None (ritorna anticipatamente se la coda è vuota o le elaborazioni risultano già in esecuzione)

Metodo: _processa_file_singolo

Firma: def _processa_file_singolo(self, item, tag_base, luogo_manuale)
Descrizione: Gestisce il ciclo vitale di importazione di un file: esegue la verifica di duplicazione record sul database locale, effettua la copia fisica del file all'interno dello storage dedicato, richiama le pipeline di intelligenza artificiale (estrazione embedding CLIP, OCR testuale, geocoding/EXIF e localizzazione facce) e salva le informazioni. Combina in un'unica entità i tag base, extra ed estratti dall'OCR.
Parametri: item (dict); tag_base (list[str]); luogo_manuale (str)
Ritorno: True (elaborazione completata) | 'dup' (rilevato duplicato) | False (errore generico riscontrato)

Metodo: _set_stato_item

Firma: def _set_stato_item(self, item, stato, testo)
Descrizione: Provvede ad aggiornare in modalità thread-safe l'etichetta testuale descrittiva dello stato di avanzamento e la colorazione visiva della riga dell'elemento specificato.
Parametri: item (dict); stato (str); testo (str)
Ritorno: None

Metodo: _set_progress

Firma: def _set_progress(self, valore)
Descrizione: Aggiorna l'indicatore grafico e la barra di avanzamento percentuale globale legata al completamento dei lavori della coda.
Parametri: valore (float tra 0.0 e 1.0)
Ritorno: None

Metodo: _aggiorna_status

Firma: def _aggiorna_status(self, messaggio)
Descrizione: Aggiorna in modalità sicura (thread-safe) la stringa testuale visualizzata nella barra informativa di stato generale del modulo.
Parametri: messaggio (str)
Ritorno: None

Metodo: _imposta_stato_elaborazione

Firma: def _imposta_stato_elaborazione(self, attivo)
Descrizione: Controlla la visibilità della barra di caricamento complessiva e provvede ad abilitare o disabilitare selettivamente l'interattività dei bottoni principali (Invio, Svuota e Applica Tag) per evitare conflitti logici.
Parametri: attivo (bool)
Ritorno: None

Metodo: _on_coda_completata

Firma: def _on_coda_completata(self, completati, saltati, errors)
Descrizione: Metodo di callback eseguito al termine dei processi: ripristina lo stato interattivo ordinario dei controlli della UI, azzera i moduli di input e la coda, comanda il rinfresco delle metriche statistiche nella schermata Home e mostra un popup riepilogativo con i conteggi degli elementi completati, saltati o falliti.
Parametri: completati (int); saltati (int); errors (int)
Ritorno: None

8.7 ricerca.py — Funzioni di modulo e classe RicercaFrame

Costanti di soglia. SOGLIA_VETTORE = SOGLIA_IMMAGINE = SOGLIA_TESTO = 85.0 (percentuale minima calibrata per mostrare un risultato). La calibrazione degli score CLIP usa i range grezzi 21.0–25.5% (testo→immagine, cross-modal) e 50.0–95.0% (immagine→immagine, same-modal).

Funzioni di modulo

Metodo: _supera_soglia

Firma: def _supera_soglia(score, score_tipo)
Descrizione: Verifica se un punteggio supera la soglia prevista per il suo tipo (sempre True per 'filtro').
Parametri: score (float); score_tipo (str): 'vettore'|'immagine'|'testo'|'filtro'
Ritorno: bool

Metodo: _calibra_score_vettore

Firma: def _calibra_score_vettore(score_grezzo)
Descrizione: Trasforma la similarità coseno grezza per ricerca testo→immagine (range 21–25.5%) in percentuale utente 0–100%.
Parametri: score_grezzo (float)
Ritorno: float (0.0–100.0, 1 decimale)

Metodo: _calibra_score_immagine

Firma: def _calibra_score_immagine(score_grezzo)
Descrizione: Trasforma la similarità coseno grezza per ricerca immagine→immagine (range 50–95%) in percentuale utente 0–100%.
Parametri: score_grezzo (float)
Ritorno: float (0.0–100.0, 1 decimale)

Metodo: _calcola_score_testo

Firma: def _calcola_score_testo(testo_query, nome_file, testo_ocr, tags)
Descrizione: Calcola la percentuale di parole della query presenti nei metadati del file e dove è avvenuto il match.
Parametri: testo_query (str); nome_file (str); testo_ocr (str); tags (str)
Ritorno: tuple (score 0–100, in_tag: bool, in_ocr: bool)

Metodi della classe RicercaFrame

Metodo: __init__

Firma: def __init__(self, parent, app)
Descrizione: Costruisce la schermata Ricerca: pannello filtri a sinistra (con riquadro immagine reference) e area risultati scrollabile a destra.
Parametri: parent (widget); app (DeepSightApp)
Ritorno: None

Metodo: re_layout

Firma: def re_layout(self, is_mobile, width)
Descrizione: Ricalcola il layout della pagina Ricerca.
Parametri: is_mobile (bool); width (int)
Ritorno: None

Metodo: avvia_ricerca

Firma: def avvia_ricerca(self)
Descrizione: Entry point: disabilita il pulsante, mostra lo spinner e avvia la ricerca in un thread separato. Applica la mutua esclusione testo/immagine reference.
Parametri: —
Ritorno: None

Metodo: _esegui_ricerca_reale

Firma: def _esegui_ricerca_reale(self)
Descrizione: Logica completa: genera l'embedding (testo o immagine), interroga ChromaDB, incrocia i filtri relazionali e la prossimità geografica su DuckDB, calibra gli score e applica la soglia; se la ricerca vettoriale è vuota ricade sul fallback testuale.
Parametri: —
Ritorno: tuple (risultati_filtrati: list[dict], esclusi: int)

Metodo: _mostra_risultati

Firma: def _mostra_risultati(self, risultati, esclusi=0)
Descrizione: Ripristina i controlli, mostra il conteggio, crea una card per risultato e riporta lo scroll in cima; placeholder se vuoto.
Parametri: risultati (list[dict]); esclusi (int)
Ritorno: None

Metodo: _crea_card_risultato

Firma: def _crea_card_risultato(self, item, row, col)
Descrizione: Costruisce la card di un risultato: thumbnail proporzionale, metadati (luogo, tag, timestamp video), badge di accuratezza, pulsante APRI e cestino 🗑.
Parametri: item (dict); row (int); col (int)
Ritorno: None

Metodo: _carica_thumbnail

Firma: def _carica_thumbnail(self, container, item)
Descrizione: Carica la miniatura reale dell'immagine (proporzionale, max 80×90 usando img.size per evitare distorsioni) o un'emoji di fallback per video/file mancanti.
Parametri: container (widget); item (dict)
Ritorno: None

Metodo: _mostra_placeholder

Firma: def _mostra_placeholder(self, testo)
Descrizione: Mostra un messaggio centrato nell'area risultati.
Parametri: testo (str)
Ritorno: None

Metodo: _svuota_risultati

Firma: def _svuota_risultati(self)
Descrizione: Rimuove tutte le card e riporta lo scroll in cima.
Parametri: —
Ritorno: None

Metodo: _scroll_to_top

Firma: def _scroll_to_top(self)
Descrizione: Riporta lo scorrimento dell'area risultati all'inizio (evita risultati “tagliati” dopo una nuova ricerca).
Parametri: —
Ritorno: None

Metodo: _mostra_prompt_reference

Firma: def _mostra_prompt_reference(self)
Descrizione: Mostra lo stato iniziale/di reset del riquadro immagine reference (testo-invito cliccabile).
Parametri: —
Ritorno: None

Metodo: _mostra_preview_reference

Firma: def _mostra_preview_reference(self, percorso)
Descrizione: Sostituisce il testo-invito con la miniatura reale, il nome file e un pulsante ✕ per rimuoverla.
Parametri: percorso (str)
Ritorno: None

Metodo: _on_text_typing

Firma: def _on_text_typing(self, event=None)
Descrizione: Se l'utente digita testo con un'immagine reference caricata, la rimuove (mutua esclusione).
Parametri: event (tk Event)
Ritorno: None

Metodo: _rimuovi_reference

Firma: def _rimuovi_reference(self)
Descrizione: Rimuove l'immagine reference e ripristina il prompt, con guardia temporizzata anti-riapertura del dialog.
Parametri: —
Ritorno: None

Metodo: seleziona_reference

Firma: def seleziona_reference(self)
Descrizione: Apre un file explorer per scegliere l'immagine di riferimento (reverse image search); protetta da guardia anti-doppia-apertura.
Parametri: —
Ritorno: None (imposta self.ricerca_img_reference)

Metodo: azzera_filtri

Firma: def azzera_filtri(self)
Descrizione: Ripristina tutti i filtri allo stato iniziale e svuota i risultati.
Parametri: —
Ritorno: None

Metodo: _apri_file

Firma: def _apri_file(self, percorso)
Descrizione: Apre il file (o la cartella che lo contiene) in Esplora File; errore se il file non esiste più.
Parametri: percorso (str)
Ritorno: None

Metodo: _on_errore_ricerca

Firma: def _on_errore_ricerca(self, messaggio)
Descrizione: Callback in caso di eccezione: ripristina i controlli e mostra l'errore.
Parametri: messaggio (str)
Ritorno: None

Metodo: _conferma_elimina

Firma: def _conferma_elimina(self, item)
Descrizione: Chiede conferma esplicita (askyesno) e, se confermato, elimina il file (uploads + DuckDB + ChromaDB), aggiorna le statistiche Home e riesegue la ricerca.
Parametri: item (dict con 'id' e 'nome_file')
Ritorno: None

9. Riferimenti

- **Python** — https://docs.python.org/3/
- **CustomTkinter** — https://github.com/TomSchimansky/CustomTkinter
- **tkinterdnd2** — https://github.com/pmgagne/tkinterdnd2
- **DuckDB** — https://duckdb.org/docs/
- **ChromaDB** — https://docs.trychroma.com/
- **PyTorch** — https://pytorch.org/docs/
- **HuggingFace Transformers / CLIP** — https://huggingface.co/docs/transformers/model_doc/clip
- **EasyOCR** — https://github.com/JaidedAI/EasyOCR
- facenet-pytorch (MTCNN, InceptionResnetV1) — https://github.com/timesler/facenet-pytorch
- **OpenCV** — https://docs.opencv.org/
- **Matplotlib** — https://matplotlib.org/stable/
- **Pillow (PIL)** — https://pillow.readthedocs.io/
- **deep-translator** — https://github.com/nidhaloff/deep-translator
- **Nominatim (OpenStreetMap)** — https://nominatim.org/release-docs/latest/

“Learning by Project - Tech, Progetto DeepSight” — documento di specifica del progetto.